Gradient Boosting

Many small trees, each fixing the last

The problem

- Tabular regression with complex interactions
- One tree: too simple or overfits
- Forests average; can we instead correct errors?

Intuition: fit the residuals

$$F_0 = \bar{y}, \qquad F_m = F_{m-1} + \eta\, h_m(x)$$

$$h_m \approx y - F_{m-1}(x)$$

- For squared loss, residual = negative gradient
- η (learning rate) shrinks each step

The boosting loop

```
def fit(self, X, y):
    X, y = np.asarray(X, float), np.asarray(y, float)
    self.f0 = float(y.mean())
    pred = np.full(len(y), self.f0)
    self.trees, self.train_loss_ = [], []
    for _ in range(self.n):
        t = RegressionTree(self.depth).fit(X, y - pred)
        pred += self.lr * t.predict(X)
        self.trees.append(t)
        self.train_loss_.append(float(np.mean((y - pred) ** 2)))
    return self
```

Regression trees with prefix sums

- Leaf value = mean of its targets
- Split cost = SSE(left) + SSE(right)
- SSE = $\Sigma y^2 - (\Sigma y)^2/n$ via cumulative sums
- O(n log n) per feature per node

Fast split search

```
for j in range(X.shape[1]):
    order = np.argsort(X[:, j])
    xs, ys = X[order, j], y[order]
    cs, cs2, n = np.cumsum(ys), np.cumsum(ys ** 2), len(ys)
    for i in range(self.min_leaf, n - self.min_leaf + 1):
        if i == n or xs[i - 1] == xs[i]:  # need a real gap between left and right
            continue
        l_sse = cs2[i - 1] - cs[i - 1] ** 2 / i
        r_sum, r_sq = cs[-1] - cs[i - 1], cs2[-1] - cs2[i - 1]
        sse = l_sse + r_sq - r_sum ** 2 / (n - i)
        if best is None or sse < best[0]:
            best = (sse, j, (xs[i - 1] + xs[i]) / 2)
```

Complexity & trade-offs

- Training: $O(M \cdot d \cdot n \log n)$ for M trees
- Sequential: harder to parallelise than forests
- Tune: learning rate, #trees, depth (early stopping)
- Usually beats forests on tabular data

Recap

- Start from the mean
- Each tree fits the current residuals
- Shrink with a learning rate
- Gradient descent in function space